

IBM Bob 2.0 Hackathon: Legacy Code X-Ray

Member 2: macOS — Execution Playbook & Prompt Reference

Target Branch: feature/code-smells-refactor	Primary Role: Stepwise Refactoring & Modularization (MVC)
GitHub Repo: subhradeepkundu270305/ibm-bob-2.0-hackathon	OS Environment: macOS / Terminal / VS Code

CRITICAL HACKATHON RULES:

- **Screenshots are Mandatory:** Screenshot every completed Bob session summary and save them to bob_sessions/<your-name>/. Judges verify these as proof of tool usage.
- **Branching Strategy:** 1 feature branch per person (feature/code-smells-refactor). Inherit Member 1's JSDocs & docstrings from main.
- **Stepwise Refactoring:** Review each diff in VS Code before accepting. Do not run all refactoring at once.
- **Automated Tests:** Ensure all tests pass with npm test before merging back to main.

1. Terminal Setup on macOS

```
# 1. Clone repository and switch to updated main:
git clone https://github.com/subhradeepkundu270305/ibm-bob-2.0-hackathon.git
cd ibm-bob-2.0-hackathon
git checkout main && git pull origin main

# 2. Create feature branch and install packages:
git checkout -b feature/code-smells-refactor
npm install

# 3. Create your personal screenshot directory (replace <name> with your name):
mkdir -p bob_sessions/<name>
```

2. Step-by-Step Prompt Execution Guide

Prompt 2.1: Code Smell Audit Verification

Goal: Catalog the 5 riskiest code smells from Member 2's session.

Prompt to Bob:

Scan this URL shortener codebase (index.js) and conduct a comprehensive code quality and smell audit.

Identify the top 5 riskiest code smells, focusing on:

1. Monolithic god file / mixed concerns
2. Copy-pasted validation logic across routes
3. Deeply nested if/else statements (pyramid of doom)
4. Magic numbers, magic strings, and lazy variable names
5. Security vulnerabilities (raw SQL concatenation) and lack of error handling

For each issue, detail:

- The exact lines/functions affected
- Why this is risky for maintenance, reliability, or security
- A step-by-step refactoring proposal

Format the output as clean Markdown so it can be saved directly as docs/code-smells.md.

Action / Screenshot: [Save to docs/code-smells.md. Screenshot -> bob_sessions/<name>/01_smell_audit.png](#)

Prompt 2.2: Refactor Smell 1 — Extract Shared Validation

Goal: Eliminate duplicate URL verification routines.

Prompt to Bob:

Refactor index.js to eliminate duplicate validation logic:

1. Extract the copy-pasted URL validation code currently repeated in POST /api/shorten and PUT /api/urls/:code into a single, clean reusable validator function validateUrl(url).
 2. Ensure it handles protocol checks, length limits, whitespace, and domain format properly.
 3. Update both routes to use this new helper.
- Keep all other functionality intact.

Action / Screenshot: [Review diff in VS Code, accept changes. Screenshot -> bob_sessions/<name>/02_refactor_validation.png](#)

Prompt 2.3: Refactor Smell 2 — Flatten Deep Nesting (Guard Clauses)

Goal: Replace pyramid of doom with early returns.

Prompt to Bob:

Refactor the route handlers in index.js to eliminate the 5-7 levels of nested if/else statements.

1. Replace nested conditionals with guard clauses and early returns.
2. Ensure error responses (e.g. 400 Bad Request) exit immediately.
3. Maintain the exact same validation criteria and status codes, but make the control flow linear and clean.

[Action / Screenshot: Review diff in VS Code, accept changes. Screenshot -> bob_sessions/<name>/03_refactor_guard_clauses.png](#)

Prompt 2.4: Refactor Smell 3 — Centralize Constants & Fix Lazy Naming

Goal: Replace hardcoded literals and cryptic variable names.

Prompt to Bob:

Refactor index.js to address naming conventions and magic values:

1. Extract all magic numbers and strings into named constants at the top of the file (PORT, MAX_URL_LENGTH, SHORT_CODE_LENGTH, BASE62_CHARSET, HTTP status codes, DB_PATH).
2. Rename all lazy, ambiguous variable names (x, res2, data, temp, tempCode, arr, arr2, thing, str, num, flag, q, obj) to standard, semantic names (req, res, targetUrl, urlRecord, shortCode, etc.).

[Action / Screenshot: Review diff in VS Code, accept changes. Screenshot -> bob_sessions/<name>/04_refactor_constants_naming.png](#)

Prompt 2.5: Refactor Smell 4 — Fix SQL Injection & Add Error Handling

Goal: Convert to parameterized queries and try/catch handlers.

Prompt to Bob:

Refactor the SQLite database interactions in index.js:

1. Replace all raw string concatenation SQL queries with parameterized statements (using ? placeholders) to eliminate SQL injection vulnerabilities.
2. Add comprehensive error handling around every database callback (db.run, db.get, db.all).
3. If an error occurs, log it and respond with a proper HTTP 500 JSON error message instead of failing silently or crashing.

[Action / Screenshot: Review diff in VS Code, accept changes. Screenshot -> bob_sessions/<name>/05_refactor_sql_security.png](#)

Prompt 2.6: Refactor Smell 5 — Modular Architecture & Tests

Goal: Split monolith into MVC structure and generate test suite.

Prompt to Bob:

Now modularize this project into a clean, maintainable structure:

1. Separate index.js into standard folders:

- src/db.js (database connection & table schema)
- src/utils/validator.js (URL & code validators)
- src/utils/shortener.js (Base62 code generator)
- src/controllers/urlController.js (CRUD business logic)
- src/routes/urlRoutes.js (Express route bindings)
- server.js (Express app initialization & listen)

2. Add a basic test suite using Node's native test runner (node:test) or Jest to verify that creating a short URL, redirecting, updating, and deleting work as expected.

3. Update package.json scripts with 'test': 'node --test'.

Action / Screenshot: Run npm test to verify. Commit to feature/code-smells-refactor. Screenshot -> bob_sessions/<name>/06_modular_tests.png

3. Verification, Commit & Merge Workflow

```
# 1. Run automated test suite to ensure 100% passes:
npm test

# 2. Check git status to confirm all files are staged:
git status

# 3. Stage and commit all refactored code and screenshots:
git add src/ server.js index.js package.json package-lock.json test/ docs/ bob_sessions/<name>/
git commit -m "refactor: modularize monolithic index.js into MVC architecture and add automated tests"

# 4. Push feature branch:
git push -u origin feature/code-smells-refactor

# 5. Merge into main so Member 3 & Member 4 can inherit the codebase:
git checkout main && git pull origin main
git merge feature/code-smells-refactor
git push origin main
```

4. Member 2 Deliverables Checklist

Item / Deliverable	Location in Repo	Required Proof / Action
Smell Audit Verification	docs/code-smells.md	bob_sessions/<name>/01_smell_audit.png
Shared Validator Helper	src/utils/validator.js	bob_sessions/<name>/02_refactor_validation.png
Flattened Guard Clauses	src/controllers/urlController.js	bob_sessions/<name>/03_refactor_guard_clauses.png
Constants & Clean Naming	src/utils/ & controllers	bob_sessions/<name>/04_refactor_constants_naming.png
Parameterized SQL & Errors	src/controllers/ & db.js	bob_sessions/<name>/05_refactor_sql_security.png
Modular MVC & Tests	server.js, src/, test/	bob_sessions/<name>/06_modular_tests.png (npm test)
Merge to main	GitHub main branch	Push to origin/main for Members 3 & 4